

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 30%

Dr.G.Ayyappan

Schema Used in All Questions:

Student (RollNo, Name, DeptID, Age, Marks) | Department (DeptID, DeptName, HOD)

Sample Data (load this into MySQL before attempting the questions):

Department

DeptID	DeptName	HOD
1	Computer Science	Dr. Rao
2	Electronics	Dr. Iyer
3	Mechanical	Dr. Nair

Student

RollNo	Name	DeptID	Age	Marks
101	Aravind	1	20	85
102	Bhavya	1	21	78
103	Chandran	1	20	90
104	Deepak	2	22	60
105	Esha	2	21	55
106	Farhan	2	22	95
107	Gowri	3	20	40
108	Hari	3	21	45
109	Indra	3	20	68

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results. Given Query: SELECT s.Name, d.DeptName FROM Student s, Department d; -- Intent: list each student with their own department's name	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly. Given Query: SELECT s.Name, s.DeptID, s.Marks FROM Student s WHERE s.Marks > (SELECT AVG(Marks) FROM Student); -- Intent: list students who score above their OWN department's average	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view. Given Query: CREATE VIEW DeptAvgMarks AS SELECT Name, DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY DeptID; SELECT * FROM DeptAvgMarks;	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation. Given Query: $\pi(\text{Name, Age})(\text{Student}) \bowtie \text{Department}$ -- Intent: natural join of Student and Department on DeptID	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints. Given Query: CREATE TABLE Student (RollNo INT PRIMARY KEY, Name VARCHAR(50), DeptID INT, Age INT, Marks INT); -- Department table only has DeptID values 1, 2, 3 INSERT INTO Student (RollNo, Name, DeptID, Age, Marks) VALUES (110, 'Arun', 9, 20, 78);	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance. Given Query:	BL6 – Create	5

	SELECT s.Name, (SELECT d.DeptName FROM Department d WHERE d.DeptID = s.DeptID) AS DeptName FROM Student s;		
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error. Given Query: SELECT DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY RollNo;	BL3 – Apply	5
8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table. Given Query: -- Intent: give 5 bonus marks to students in Department 2 only UPDATE Student SET Marks = Marks + 5;	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes. Given Query: SELECT s.Name, s.Marks, (SELECT DeptName FROM Department WHERE DeptID = s.DeptID) AS DeptName, (SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS HeadOfDept FROM Student s WHERE s.DeptID IN (SELECT DeptID FROM Department WHERE DeptName IN ('Computer Science', 'Electronics'));	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage. Given Query: -- Intent: students who scored more than AT LEAST ONE student in Department 2 { s.Name Student(s) $\wedge$ ($\forall$ t) (Student(t) $\wedge$ t.DeptID = 2 $\rightarrow$ s.Marks > t.Marks) }	BL3 – Apply	5

Code of Conduct:

*I DARA KUSUMANJALI (192425398L) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

ANSWERS

1. identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.

Given Query:

```
SELECT s.Name, d.DeptName
FROM Student s, Department d;
-- Intent: list each student with their own department's name
```

Correct Query using INNER JOIN

```
SELECT s.Name, d.DeptName
FROM Student s INNER JOIN Department d ON s.DeptID = d.DeptID;
```

Explanation

- Student s → gives the student details.
- Department d → gives the department details.
- ON s.DeptID = d.DeptID → matches each student with **their own department**.
- INNER JOIN → returns only students whose department exists in the Department table.

Error: Missing JOIN condition.

Fix: Use INNER JOIN ... ON s.DeptID = d.DeptID

2. The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.

Given Query:

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks > (SELECT AVG(Marks) FROM Student);
-- Intent: list students who score above their OWN department's average
```

Correct correlated subquery

```

SELECT s.Name, s.DeptID, s.Marks FROM Student s WHERE s.Marks > (
    SELECT AVG(s2.Marks)
    FROM Student s2
    WHERE s2.DeptID = s.DeptID
);

```

What was fixed?

- $s \rightarrow$ represents the current student from the outer query.
- $s2 \rightarrow$ represents students considered inside the subquery.
- $s2.DeptID = s.DeptID \rightarrow$ **correlates the subquery with the outer query.**
- $AVG(s2.Marks) \rightarrow$ calculates the average **only for that student's department.**

Result: Students are listed only when their marks are greater than the **average marks of their own department.**

3.A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.

Given Query:

```

CREATE VIEW DeptAvgMarks AS
SELECT Name, DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;

```

```

SELECT * FROM DeptAvgMarks;

```

Correct and optimized view

```

CREATE VIEW DeptAvgMarks AS
SELECT DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
SELECT * FROM DeptAvgMarks;

```

Why this is correct

- DeptID identifies each department.
- AVG(Marks) calculates the average marks for that department.
- GROUP BY DeptID groups students department-wise.

- Name is removed because an individual student's name has no meaning in a department-level average.

Error: Name is neither aggregated nor included in GROUP BY.

Fix: Remove Name from the SELECT list.

4. Debug the Relational Algebra Expression for Natural Join

Debug the given relational algebra expression that produces incorrect output for a natural join operation.

Given Query:

$\pi(\text{Name, Age}) (\text{Student}) \bowtie \text{Department}$

-- Intent: natural join of Student and Department on DeptID

Given Query

$\pi(\text{Name, Age}) (\text{Student}) \bowtie \text{Department}$

Error

The given expression performs **projection on the Student relation before the natural join**.

The projection:

$\pi(\text{Name, Age}) (\text{Student})$

retains only Name and Age attributes and removes DeptID.

However, the natural join between Student and Department is intended to be performed using the common attribute **DeptID**. Since DeptID has already been removed, the natural join cannot correctly match students with their departments.

Correct Relational Algebra Expression

$\pi(\text{Name, Age}) (\text{Student} \bowtie \text{Department})$

Explanation

First, the Student and Department relations are naturally joined:

$\text{Student} \bowtie \text{Department}$

A natural join automatically matches the common attribute, which in this case is:

Student.DeptID = Department.DeptID

After obtaining the joined relation, projection is applied to retrieve only the required attributes:

$\pi(\text{Name, Age})$

Therefore, the correct order is:

Student $\rightarrow$ Natural Join with Department $\rightarrow$ Projection

Example

Suppose Student contains:

RollNo Name DeptID Marks

101 Arun 1 78

102 Priya 2 85

and Department contains:

DeptID DeptName

1 Computer Science

2 Electronics

The natural join matches the records based on DeptID.

The final projection produces:

Name Age

Arun ...

Priya ...

Conclusion

The main error is **projecting away DeptID before performing the join**. The join must be performed first, followed by projection.

5. Referential Integrity Violation

The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.

Given Query:

```
CREATE TABLE Student (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DeptID INT,  
    Age INT,  
    Marks INT  
);  
  
-- Department table only has DeptID values 1, 2, 3  
  
INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)  
VALUES (110, 'Arun', 9, 20, 78);
```

Given Query

```
CREATE TABLE Student (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DeptID INT,  
    Age INT,  
    Marks INT  
);  
  
INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)VALUES (110, 'Arun', 9, 20, 78);
```

The Department table contains only:

DeptID = 1, 2, 3

Error

The student record contains:

DeptID = 9

But there is no department with DeptID = 9 in the Department table.

This creates a **referential integrity violation**.

A student should belong only to a department that actually exists in the Department table.

Correct Table Design

The Department table should have DeptID as its primary key:

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50)  
);
```

The Student table should contain a foreign key referencing Department(DeptID):

```
CREATE TABLE Student (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DeptID INT,  
    Age INT,  
    Marks INT,  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

Correct Insertion

Since the available department IDs are 1, 2, 3, the student must use one of these valid IDs.

```
INSERT INTO Student (RollNo, Name, DeptID, Age, Marks) VALUES (110, 'Arun', 2, 20, 78);
```

How the Foreign Key Works

The constraint:

FOREIGN KEY (DeptID) REFERENCES Department(DeptID)

ensures that every DeptID inserted into Student must already exist in the Department table.

For example:

DeptID = 2

is valid because Department 2 exists.

But:

DeptID = 9

is rejected because Department 9 does not exist.

Conclusion

The problem is caused by inserting a **non-existent department ID**. Defining DeptID as a foreign key prevents invalid department references and maintains **referential integrity**.

6. Optimize the Query Using JOIN

Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.

Given Query:

```
SELECT s.Name,  
       (SELECT d.DeptName FROM Department d WHERE d.DeptID = s.DeptID) AS DeptName  
FROM Student s;
```

Given Query

```
SELECT s.Name,  
       (SELECT d.DeptName  
        FROM Department d  
        WHERE d.DeptID = s.DeptID) AS DeptName  
FROM Student s;
```

Problem

The query uses a **correlated subquery**.

For every student in the Student table, the database must execute the inner query to find the corresponding department.

Conceptually, if there are 1,000 students, the database may need to perform the department lookup repeatedly for those rows.

This makes the query unnecessarily complicated and can reduce performance, especially with large datasets.

Optimized Query

```
SELECT s.Name,  
       d.DeptName  
FROM Student s  
INNER JOIN Department d  
ON s.DeptID = d.DeptID;
```

Explanation

The JOIN directly connects the two tables using:

```
ON s.DeptID = d.DeptID
```

This tells the database that each student's DeptID should match the corresponding department's DeptID.

The query retrieves:

- s.Name → student's name
- d.DeptName → department name

Example

Suppose:

Student

RollNo	Name	DeptID
101	Arun	1
102	Priya	2

Department

DeptID	DeptName
1	Computer Science
2	Electronics

The JOIN produces:

Name	DeptName
Arun	Computer Science
Priya	Electronics

Performance Advantage

The JOIN version is generally preferred because:

1. It clearly expresses the relationship between tables.
2. It avoids repeated correlated lookups.
3. The database optimizer can choose an efficient join strategy.
4. Appropriate indexes can make the join very fast.
5. The query is easier to read and maintain.

Conclusion

The correlated subquery should be replaced with an INNER JOIN when retrieving related information from another table.

7.Debug the GROUP BY Query

The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.

Given Query:

```
SELECT DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY RollNo;
```

Given Query

```
SELECT DeptID, AVG(Marks) AS AvgMarksFROM StudentGROUP BY RollNo;
```

Error

The query selects:

DeptID

and calculates:

AVG(Marks)

but groups the records using:

GROUP BY RollNo

This is logically incorrect because the requirement is to calculate the **average marks for each department**, not for each student.

RollNo identifies an individual student, whereas DeptID identifies a department.

Correct Query

```
SELECT DeptID,
       AVG(Marks) AS AvgMarksFROM StudentGROUP BY DeptID;
```

Explanation

The GROUP BY clause determines how records are divided into groups.

If we use:

GROUP BY RollNo

each student is generally treated as a separate group.

For example:

RollNo	DeptID	Marks
101	1	80
102	1	90
103	2	70
104	2	80

Grouping by RollNo does not calculate the department average.

Instead, grouping by DeptID produces:

Department 1:

$$(80 + 90) / 2 = 85$$

Department 2:

$$(70 + 80) / 2 = 75$$

Therefore, the output is:

DeptID	AvgMarks
1	85
2	75

Conclusion

Since the aggregation is required **department-wise**, DeptID must be used in the GROUP BY clause.

Incorrect:

GROUP BY RollNo

Correct:

GROUP BY DeptID

8. Debug the UPDATE Statement- The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.

Given Query:

```
SELECT DeptID, AVG(Marks) AS AvgMarks  
FROM Student  
GROUP BY RollNo;
```

Given Query

```
UPDATE StudentSET Marks = Marks + 5;
```

Intent

The intention is:

Give 5 bonus marks only to students belonging to Department 2.

Error

The query does not contain a WHERE clause.

In SQL, an UPDATE statement without a WHERE condition applies to **all rows** in the table.

Therefore, the given query will increase the marks of every student by 5, regardless of their department.

Correct Query

```
UPDATE StudentSET Marks = Marks + 5WHERE DeptID = 2;
```

Explanation

The SET clause:

SET Marks = Marks + 5

increases the existing marks by 5.

The WHERE clause:

WHERE DeptID = 2

restricts the update to students belonging to Department 2.

Example

Before update:

Name	DeptID	Marks
Arun	1	70
Priya	2	80
Ravi	2	75
Meena	3	90

After:

UPDATE StudentSET Marks = Marks + 5WHERE DeptID = 2;

the result becomes:

Name	DeptID	Marks
Arun	1	70
Priya	2	85
Ravi	2	80
Meena	3	90

Only Department 2 students are modified.

Important Point

Before executing an UPDATE, it is good practice to test the condition using:

```
SELECT *FROM StudentWHERE DeptID = 2;
```

This confirms which rows will be affected.

Conclusion

The logical error is the **missing** WHERE **clause**. Always use an appropriate condition when only selected rows should be modified.

9. Optimize the Multi-Table Query

Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.

Given Query:

```
SELECT s.Name, s.Marks,  
      (SELECT DeptName FROM Department WHERE DeptID = s.DeptID) AS DeptName,  
      (SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS HeadOfDept  
FROM Student s  
WHERE s.DeptID IN (  
      SELECT DeptID FROM Department WHERE DeptName IN ('Computer Science', 'Electronics')  
);
```

Given Query

```
SELECT s.Name, s.Marks,  
      (SELECT DeptName  
      FROM Department  
      WHERE DeptID = s.DeptID) AS DeptName,  
      (SELECT HOD  
      FROM Department  
      WHERE DeptID = s.DeptID) AS HeadOfDeptFROM Student sWHERE s.DeptID IN (  
      SELECT DeptID
```

```
FROM Department
WHERE DeptName IN ('Computer Science', 'Electronics')
);
```

Problems with the Query

The query uses multiple subqueries to access the Department table.

There are two correlated subqueries:

```
SELECT DeptName ...
```

and

```
SELECT HOD ...
```

Both repeatedly search the Department table.

There is also another subquery in the WHERE clause to identify the required departments.

This makes the query unnecessarily complex.

Optimized Query

```
SELECT s.Name,
       s.Marks,
       d.DeptName,
       d.HOD AS HeadOfDept
FROM Student s INNER JOIN Department d
ON s.DeptID = d.DeptID
WHERE d.DeptName IN ('Computer Science', 'Electronics');
```

Explanation

The JOIN directly connects the two tables:

```
ON s.DeptID = d.DeptID
```

The department name and HOD can then be retrieved directly:

```
d.DeptName
```

d.HOD

The filtering condition is applied directly to the department table:

```
WHERE d.DeptName IN ('Computer Science', 'Electronics')
```

This eliminates the unnecessary nested subqueries.

Recommended Indexes

An index can be created on Student.DeptID:

```
CREATE INDEX idx_student_deptid ON Student(DeptID);
```

An index can also be created on Department.DeptName:

```
CREATE INDEX idx_department_name ON Department(DeptName);
```

The Department.DeptID column should already be indexed if it is the primary key.

Advantages

The optimized query:

- Uses a direct JOIN.
- Removes multiple correlated subqueries.
- Improves readability.
- Gives the query optimizer more flexibility.
- Can benefit from indexes on join/filter columns.
- Avoids repeatedly searching the same department records.

Conclusion

The best approach is to replace repeated subqueries with a single INNER JOIN and apply the department filtering directly.

10. Debug the Relational Calculus Expression

Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.

Given Query:

-- Intent: students who scored more than AT LEAST ONE student in Department 2

$\{ s.Name \mid Student(s) \wedge (\forall t) (Student(t) \wedge t.DeptID = 2 \rightarrow s.Marks > t.Marks) \}$

Given Expression

```
{ s.Name |  
  Student(s)  $\wedge$   
  ( $\forall t$ ) (Student(t)  $\wedge$  t.DeptID = 2  $\rightarrow$  s.Marks > t.Marks)  
}
```

Intent

The requirement is:

Find students who scored more than **at least one** student in Department 2.

Error

The expression uses the **universal quantifier**:

$\forall t$

The symbol $\forall$ means "**for all**".

Therefore, the given expression actually asks for students whose marks are greater than the marks of **every student in Department 2**.

That is different from the intended requirement.

The intended phrase is:

at least one student

For "at least one", we need the **existential quantifier**:

$\exists$

which means "**there exists**".

Correct Expression

```
{ s.Name |  
  Student(s)  $\wedge$   
  ( $\exists$  t) (  
    Student(t)  $\wedge$   
    t.DeptID = 2  $\wedge$   
    s.Marks > t.Marks  
  )  
}
```

Explanation

The expression:

$(\exists t)$

means that there must exist **at least one student** t satisfying the conditions.

The conditions are:

Student(t)

t must be a student.

$t.$ DeptID = 2

t must belong to Department 2.

$s.$ Marks > $t.$ Marks

The marks of student s must be greater than the marks of that Department 2 student.

Therefore, if even one Department 2 student has lower marks than s , the student s satisfies the condition.

Example

Suppose Department 2 students have:

Student	Marks
Ravi	60
Priya	75
Kiran	85

Suppose Arun has:

Arun = 70

Arun scored higher than Ravi (60), so Arun satisfies the "**at least one**" condition.

However, Arun did not score higher than Priya (75) or Kiran (85).

Therefore:

- $\exists \rightarrow$ Arun qualifies.
- $\forall \rightarrow$ Arun does not qualify.

Difference Between $\exists$ and $\forall$

Quantifier	Meaning	Requirement
$\exists$	There exists	At least one student
$\forall$	For all	Every student

Conclusion

The logical error is the use of:

$\forall$

instead of:

$\exists$

The corrected expression is:

```
{ s.Name |  
  Student(s) ∧  
  (∃ t) (  
    Student(t) ∧  
    t.DeptID = 2 ∧  
    s.Marks > t.Marks  
  )  
}
```

This correctly identifies students who scored higher than **at least one student in Department 2.**